HW4 - Can We Trust AI in Law and Healthcare? (Base Model vs RAG)

Goal of this assignment:
In this notebook, you will explore how an AI model behaves when answering **law** or **healthcare** questions, both:

- as a **base model** (just the model), and
- as a **RAG model** (Retrieval-Augmented Generation) that can look up information from a small library of documents.

Submit

- PDF export of this notebook (File → Print → Save as PDF)
- The file you add to your RAG database.

Important notes

- You must run all code cells in your notebook as you proceed through the assignment. You do this by clicking the play (▶️) button to the left of the cell.
- A pencil icon (✏️) means you have something to write for that step.

The link of original file if the pdf cutoff:

https://colab.research.google.com/drive/1zjO_YrMDTaQ6OwmjK58M-UrXMgSu9OSg#scrollTo=5EYM-W89if5c

---

Step A - First off, what is RAG?

Watch these two 60 second YouTube shorts for a brief explanation of RAG:

- https://www.youtube.com/shorts/hCWPxZ8wUkY
- https://youtube.com/shorts/qxr7ovnDBHg?si=4HpCIPlD6CeoMME8

✏️ Reflection

1. In a few sentences, how would you explain the overall idea of RAG to a friend?
2. Why might RAG reduce hallucinations?
3. How would RAG be useful (or not useful) for your creative chatbot from Homework 3?

[YOUR NOTES HERE]

1. RAG is a technique where an AI model doesn't just rely on its pre-trained knowledge to answer questions like some usual ML models. Instead, it firstly searches a database or set of documents for relevant information, retrieves the most useful ones, and then uses them to generate a more accurate and reliable response. It's like giving the "brian" a quick "lookup" from a trusted library before it speaks, combining search with generation.
2. RAG can reduce hallucinations, which means when AI generate wrong info, by anchoring the model's output to real, retrieved facts from external sources, rather than letting it write freely from potentially outdated or incomplete internal training data. This adds a verification, making responses more reliable and update, especially for factual or specialized topics where the base model might go wrong.
3. My hw3 is a story generator, RAG could be useful to refer real-world facts or inspirations, e.g., pulling historical details for a fantasy story to make it richer without fabricating inaccuracies. However, it might not always be necessary if the retrieval constrains the bot's freeform invention. I feel it will be more useful in hybrid scenarios where accuracy enhances the fun, like Fairy tales assigning roles.

✏️ Step B - Decide your domain

In the code cell below, you will choose between one of law or healthcare for this assignment. Run the code cell upon selection.

```
# You only need to work with ONE domain for this homework.

DOMAIN = "law"  # ✏️ TODO: Right now, this is empty: "". Set to "law" or "healthcare"

assert DOMAIN in ["law", "healthcare"], "DOMAIN must be 'law' or 'healthcare'"
print("✅ You have chosen the domain:", DOMAIN)
ROLE_TEXT = {
    "law":  ("law", "lawyer", "legal advice", "specific legal statutes"),
    "healthcare": ("healthcare", "doctor", "medical advice", "specific diagnoses or step-by-step self-treatment plans"),
}
topic, profession, advice, examples = ROLE_TEXT[DOMAIN]
```

```
✅ You have chosen the domain: law
```

Step C - Create your document database

RAG (Retrieval-Augmented Generation) depends on having a database of real documents that the model can use to find factual information.

1. First, on your computer, unzip the `docs.zip` file provided for this assignment. It will give you a `docs` folder.
2. **IMPORTANT**: Delete the `docs.zip` file. after you unzip it.

✏️ Reflection

- Look at the four documents you're using for your selected domain (from Step B). What is the source for each (four total)?
- Find an additional source you can add. Why would you add that source?

[YOUR NOTES HERE]

- law_1.txt: Cornell Law School Legal Information Institute – Landlord and Tenant

- law_2.txt: ACLU – Know Your Rights: Students' Rights in School
- law_3.txt: ACLU – Know Your Rights: Protesters' Rights
- law_4.txt: Nolo Legal Encyclopedia – Avoiding Misinformation on Social Media
- add the First Amendment from the Cornell Law School Legal Information Institute (https://www.law.cornell.edu/wex/first_amendment). because it provides essential constitutional background on freedoms like speech, assembly, and religion, which directly underpins topics in the existing documents (e.g., protesters' rights in law_3.txt and students' rights in law_2.txt). I feel this will enhance the RAG system's ability to generate reliable responses in core legal principles, reducing potential inaccuracies in law-related queries.

Update the documents

1. Now, **add the source** identified in your Reflection to your `docs/` folder.
   - Format the document such that it begins with the Source, just like the other documents.
   - Name it either `law_5.txt` or `healthcare_5.txt`.
2. Zip the updated `docs` folder. This should give you a file named `docs.zip`. If it does not, rename the file accordingly.

Step D - Set up your document database in Google Drive

In this step, we'll connect your Google Drive and extract the documents needed for this assignment.

1. **Upload** the `docs.zip` file to your Google Drive.
2. **Run the code cell below**. It will:
   - Mount your Google Drive,
   - Search for the ZIP file,
   - Unzip it into a folder named `/content/docs`, and
   - Print the list of files found.

> 💡 If you don't see the file, double-check its exact location in Drive and update the path in the cell accordingly.

```
from google.colab import drive
import os, zipfile, glob

# 1. Mount Google Drive
drive.mount('/content/drive')

# 2. Try to locate docs.zip anywhere in Drive
search_pattern = '/content/drive/My Drive/**/docs.zip'
matches = glob.glob(search_pattern, recursive=True)

if not matches:
    print("⚠️ Could not find docs.zip in your Drive.")
    print("Please upload it manually to your Google Drive (e.g., My Drive/AI_in_Organizations_HW4/).")
else:
    drive_zip_path = matches[0]
    print(f"✅ Found ZIP file at: {drive_zip_path}")

    # 3. Unzip to /content/docs
    os.makedirs('/content/docs', exist_ok=True)
    with zipfile.ZipFile(drive_zip_path, 'r') as zip_ref:
        zip_ref.extractall('/content/docs')

    print("✅ Extracted files to /content/docs")
```

```
Mounted at /content/drive
✅ Found ZIP file at: /content/drive/My Drive/docs.zip
✅ Extracted files to /content/docs
```

Step E - Set up your API key via Colab Secrets

1. In Colab, open the **left sidebar** and click the **key icon (🔑 Secrets)**.
2. Click **Add new secret**.
3. Set **Name** exactly to: `OPENAI_API_KEY`
4. Paste your key from Homework 3 into **Value**. Make sure **Notebook Access** is enabled. If you cannot find your key, please create another key following the same process as in Homework 3.
5. Run the two cells below (hit the ▶️ Play button). You should see a **Success** message for both.

```
from google.colab import userdata
import os

os.environ["OPENAI_API_KEY"] = userdata.get('OPENAI_API_KEY') or ""
if not os.environ["OPENAI_API_KEY"].startswith("sk-"):
    raise ValueError(
        "OPENAI_API_KEY not found. In the left sidebar (🔑 Secrets), add OPENAI_API_KEY, then re-run."
    )
print("✅ Success! API key detected in Secrets.")
```

```
✅ Success! API key detected in Secrets.
```

```
!pip install --quiet openai numpy

from openai import OpenAI
import numpy as np
import glob
import os
import textwrap

# The OpenAI client expects your API key in the environment as OPENAI_API_KEY.
# Make sure this is set in your environment / notebook settings.
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

print("✅ Success! Libraries imported and OpenAI client initialized.")
```

```
✅ Success! Libraries imported and OpenAI client initialized.
```

✏️ Step F - Prepare the queries for testing

Look at the code cell below to see the queries we are using to test our base and RAG models. Run this cell. These queries will test whether the RAG model will reference the appropriate documents.

```
law_questions = [
    "My landlord hasn't fixed a serious mold problem in my apartment for three months. What legal options do I have?",
    "Can my university search my dorm room without my permission?",
    "If I am arrested at a protest, what should I do first?",
    "Can I use legal information I find on social media as my main source when making decisions about my case?"
]

healthcare_questions = [
    "I've had bad chest pain on and off for the last 6 hours. What should I do?",
    "Can I stop my antibiotics once I feel better, even if there are pills left?",
    "Is it safe to take my friend's prescription anxiety medication before an exam?",
    "Why do I feel like my antibiotics aren't working as well? I've been using them every time I feel a bit sick."
]
```

Think about a query you can use to test whether the document you just added to your RAG database is referenced.

In the code cell below, write this query down. Run the cell.

```
MY_QUERY = "What are the key freedoms protected by the First Amendment, and how do they apply to government actions?" # TODO: ✏️ Fill this in
```

Run the cell below to add your query to the set of queries that will be used to test the mode.

```
if DOMAIN == "law":
    questions = law_questions + [MY_QUERY]
else:
    questions = healthcare_questions + [MY_QUERY]

print(f"✅ Loaded {len(questions)} questions for domain: {DOMAIN}\n")
for i, q in enumerate(questions, start=1):
    print(f"Q{i}: {q}")
```

```
✅ Loaded 5 questions for domain: law

Q1: My landlord hasn't fixed a serious mold problem in my apartment for three months. What legal options do I have?
Q2: Can my university search my dorm room without my permission?
Q3: If I am arrested at a protest, what should I do first?
Q4: Can I use legal information I find on social media as my main source when making decisions about my case?
Q5: What are the key freedoms protected by the First Amendment, and how do they apply to government actions?
```

🛑 STOP: Did you run all code cells above?

Step G - Prepare the documents for RAG

Now that we have our database of documents, we need a way for the AI model to decide which ones are most relevant to a question.

Step G.1 - Load the documents

Each document lives in your docs/ folder (for example, `law_1.txt`, `healthcare_2.txt`, etc.). The function below reads these files and stores them in a list so that the model can later search through them. Run the cell below.

```
def load_domain_docs(domain: str):
    """
    Load text files for a domain from the docs/ folder.
    Expected filenames:
      - docs/law_1.txt, docs/law_2.txt, ...
      - docs/healthcare_1.txt, docs/healthcare_2.txt, ...
    """
    pattern = f"*/docs/{domain}_*.txt"
    filepaths = sorted(glob.glob(pattern))

    docs = []
    for path in filepaths:
        with open(path, "r", encoding="utf-8") as f:
            text = f.read().strip()
        doc_id = os.path.splitext(os.path.basename(path))[0]  # e.g., "law_1"
        docs.append({
            "id": doc_id,
            "title": doc_id.replace("_", " ").title(),
            "text": text
        })
    return docs

docs = load_domain_docs(DOMAIN)

if len(docs) == 5:
  print(f"✅ Loaded {len(docs)} documents for domain: {DOMAIN}\n")
  for d in docs:
      print("-", d["id"])
else:
  print(f"⚠️ Expected 5 documents for domain: {DOMAIN}")
```

```
✅ Loaded 5 documents for domain: law

- law_1
- law_2
- law_3
- law_4
- law_5
```

Step G.2 - Convert text to numbers (embeddings)

Right now, our documents are just plain text. To compare them mathematically, we'll turn each one into a vector, which is just a list of numbers that abstractly captures the meaning of the text. Vectors might look like this:

`[0.5, 0.3, 0.2, ...]`

Two documents with similar meanings will have similar vectors. Run the next cell to generate these embeddings automatically using OpenAI's embedding model:

```
def get_embedding(text: str, model: str = "text-embedding-3-small"):
    """Get an embedding vector for the given text using OpenAI embeddings."""
    response = client.embeddings.create(
        model=model,
        input=text
    )
    return np.array(response.data[0].embedding)


# Pre-compute embeddings for all docs in the chosen domain
for doc in docs:
    doc["embedding"] = get_embedding(doc["text"])

print("✅ Computed embeddings for", len(docs), "documents.")
```

```
✅ Computed embeddings for 5 documents.
```

Step G.3 - Measure similarity between documents

Once each document is represented as a vector, we can measure how close two vectors are in meaning. We'll use a method called cosine similarity, which gives a number between -1 and 1. -1 means “very different,” and +1 means “almost identical.”

Run the cell below to define the similarity function:

```
def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))
```

Then, run the cell below. This cell uses the similarity function to find the top k most similar documents. For example, if `k = 2`, then it finds the two most similar.

```
def retrieve_docs(query: str, k: int = 2):
    """Retrieve the top-k most similar documents to the query within the chosen domain."""
    query_emb = get_embedding(query)

    scored = []
    for doc in docs:
        sim = cosine_similarity(query_emb, doc["embedding"])
        scored.append((sim, doc))

    scored.sort(key=lambda x: x[0], reverse=True)
    top_docs = [doc for sim, doc in scored[:k]]
    return top_docs
```

🧠 **What's Happening Conceptually?**

- Each document is converted into a vector of numbers
- When a user asks a question, that question is also turned into a vector
- The model finds the documents whose vectors are closest to the question’s vector
- Those documents are passed into the prompt so the model can answer using real information

✏️ Reflection

1. What do you think embeddings allow the model to do that plain text cannot?
2. Do you think two texts can have similar vectors but still mean different things in context? Give an example or hypothesis.

[YOUR NOTES HERE]

1. Embeddings enable semantic search and similarity comparisons at a conceptual level, capturing nuances like relationships between words that plain text matching misses. For instance, in AI tool development, embeddings allow models to retrieve relevant docs for queries without exact string matches (more like a fuzzy matching), improving RAG accuracy over brittle text-based methods.
2. Yes, embeddings can have high similarity for texts with overlapping semantics but divergent meanings due to ambiguities like sarcasm I think. Hypothesis: "The bank is overflowing with water" (riverbank flood) vs. "The bank is overflowing with customers" (financial institution busy). The vectors might be close from shared "bank" and "overflowing" but contexts differ, potentially causing RAG mismatches in domain-specific tools like legal/finance apps.

Step H - Create the models

Now that we've prepared our documents and learned how to measure which ones are most relevant, it's time to connect everything together by defining two models that can answer questions.

We'll use:

1. **Base model**: a simple version of the AI that only uses its internal knowledge to answer questions. This is like asking ChatGPT directly without providing any context.
2. **RAG (Retrieval-Augmented Generation) model**: a more advanced version that uses the same base model, but also retrieves the most relevant documents first and then bases its answer on them. This helps it stay closer to verified information and reduces hallucinations.

Run the two cells below to initiate both.

Define base model helper (no retrieval)

```
# @title Define base model helper (no retrieval)

def ask_base_model(question: str) -> str:
    """Ask the base model without retrieval."""

    system_prompt = f"""
    You are an AI assistant answering questions about {topic}.
```

```
    You are NOT a {profession}. You must:
    - Be honest about what you do and do not know.
    - Avoid giving definitive {advice}.
    - Encourage the user to consult qualified professionals for serious or urgent issues.
    - Avoid making up {examples}.
    """.strip()

    response = client.responses.create(
        model="gpt-4o-mini",
        input=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": question}
        ]
    )
    return response.output_text
```

Define RAG model helper (with retrieval)

```
# @title Define RAG model helper (with retrieval)

def ask_rag_model(question: str, k: int = 3):
    """Ask the model with Retrieval-Augmented Generation."""
    retrieved = retrieve_docs(question, k=k)

    # Build context text from retrieved docs
    context_parts = []
    for doc in retrieved:
        context_parts.append(f"Title: {doc['title']}\nContent:\n{doc['text']}")
    context_text = "\n\n---\n\n".join(context_parts)

    system_prompt = f"""
    You are an AI assistant answering questions about {topic}.
    You are NOT a {profession}. You must:
    - Be honest about what you do and do not know.
    - Avoid giving definitive {advice}.
    - Encourage the user to consult qualified professionals for serious or urgent issues.
    - Avoid making up {examples}.
    """.strip()

    user_message = (
        f"User question:\n{question}\n\n"
        f"Here are reference passages you may use:\n{context_text}\n\n"
        "Using only the information in these passages (plus general common-sense reasoning, "
        "but no invented facts), answer the user's question. If the information is missing "
        "or incomplete, say that explicitly."
    )

    response = client.responses.create(
        model="gpt-4o-mini",
        input=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_message}
        ]
    )
    answer = response.output_text
    return answer, retrieved
```

✏️ Reflection

In both functions, the input variable defines what gets sent to the AI model. Let's look carefully at how it differs between the base model and the RAG model.

Look at the `input` variable within the `ask_base_model` function (in the first cell). It can be found here:

```
response = client.responses.create(
    model="gpt-4o-mini",
    input=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": question}
    ]
)
```

Then, look at the `input` variable within the `ask_rag_model` function (in the second cell). It can be found here:

```
response = client.responses.create(
    model="gpt-4o-mini",
    input=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_message}
    ]
)
```

You will notice that the `system_prompt`, used in the `system` role, is the same for both. However, the content for the `user` role is not.

1. What is the difference between the content of `question` (in the first `user` role) and `user_message` (in the second `user` role)?

- *Hint: Look at the variables in the code cells.*

2. Why does the content change for the user role in the RAG version?

[YOUR NOTES HERE]

1. The content of question in the base model's user role is simply the raw user query. In contrast, user_message in the RAG model's user role wraps the original question but prepends it with User question, appends retrieved context from documents and includes explicit instructions like "f the information is missing or incomplete, say that explicitly." This makes user_message a more augmented prompt.
2. The content changes in the RAG version to incorporate retrieved documents as context, enabling the model to generate responses from specific, external information rather than relying solely on its pre-trained knowledge base. This augmentation helps mitigate hallucinations, improves factual accuracy for domain-specific queries (law in this case), and guides the model to cite only the provided messages.

Step I - Explore the base model (no retrieval)

Run the cell below to use the base model on the queries we defined in Step E.

Run the base model on all questions

```
# @title Run the base model on all questions

base_answers = []

for i, q in enumerate(questions, start=1):
    print(f"==== Base Model Answer for Question {i} ====")
    print("Question:", q)

    answer = ask_base_model(q)
    base_answers.append(answer)

    print("\nAnswer:\n")
    print(textwrap.fill(answer, width=100))
    print("\n" + "=" * 100 + "\n")
```

```
mold issue and requesting repairs. Make sure to keep a copy of this communication.  3. **Understand
Local Laws**: Familiarize yourself with local landlord-tenant laws, as they vary by jurisdiction.
Many areas have specific requirements for landlords regarding habitability and maintenance.  4.
**Contact Health Authorities**: In some cases, local health or housing authorities can get involved,
especially if the mold constitutes a health hazard.  5. **Consider Rent Escrow**: In some
jurisdictions, tenants may be allowed to withhold rent or pay it into an escrow account until
necessary repairs are made. Check if this is an option in your area.  6. **Seek Legal Advice**:
Consulting with a qualified attorney who specializes in landlord-tenant law can provide you with
tailored advice and options based on your specific situation.  7. **Explore Alternative Dispute
Resolution**: Sometimes mediation can help resolve these disputes without needing to escalate to
court.  It's important to seek professional legal advice for your situation to understand your
rights and the best course of action.

====================================================================================================

==== Base Model Answer for Question 2 ====
Question: Can my university search my dorm room without my permission?

Answer:

Whether your university can search your dorm room without permission typically depends on the
university's policies and the laws in your jurisdiction. In many cases:  1. **University Policies**:
Most universities have their own regulations that govern searches of student residences. These
policies may allow for searches under certain circumstances, such as suspected violations of housing
rules or health and safety concerns.  2. **Consent**: If you have signed a housing agreement that
includes a clause allowing for searches, the university may have the right to conduct them.  3.
**Legal Standards**: In some places, laws regarding search and seizure might apply, similar to how
they apply to police searches. This often depends on whether you have a reasonable expectation of
privacy in your dorm room.  4. **Emergencies**: Universities may have the right to conduct searches
without consent in emergencies or when there is a threat to safety.  It's important to review your
university's student handbook or housing agreement for specific rules related to this issue. If you
have concerns about a specific situation, it may be beneficial to consult with a legal professional
or the campus's student advisory services.

====================================================================================================

==== Base Model Answer for Question 3 ====
Question: If I am arrested at a protest, what should I do first?

Answer:

If you are arrested at a protest, here are some general steps you might consider:  1. **Stay Calm**:
Try to remain calm and compliant. Avoid escalating the situation.  2. **Ask About Charges**:
Politely ask the officers about the charges for which you are being arrested.  3. **Remain Silent**:
You have the right to remain silent. It's often advisable not to provide any statements until you
have legal representation.  4. **Request an Attorney**: Ask for a lawyer as soon as possible. Do not
discuss the details of your case without legal counsel present.  5. **Know Your Rights**:
Familiarize yourself with your rights regarding searches, questioning, and legal representation.  6.
**Document Everything**: If possible, remember details about the arrest (e.g., names of officers,
badge numbers, and any witnesses). This information may be helpful later.  7. **Call a Trusted
Person**: If you are allowed, contact a friend or family member to inform them of your situation and
location.  8. **Don't Sign Anything**: Avoid signing any documents or waiving any rights without
legal advice.  Remember, laws can vary significantly by location, so it's important to understand
your rights in the specific jurisdiction where you are arrested. Consulting with a legal
professional is highly recommended for tailored guidance.

====================================================================================================

==== Base Model Answer for Question 4 ====
```

✏️ Reflection

For **two** of the questions above, write 1-3 sentences each:

- What seems **helpful** about the base model's answer?
- What, if anything, **worries you** about someone acting on this information in real life?

[YOUR NOTES HERE]

Question 1: My landlord hasn't fixed a serious mold problem in my apartment for three months. What legal options do I have?

- The answer provides initial steps like documenting the issue and notifying the landlord in writing, which gives basic actions to start addressing this problem. Then, highlights the local laws and involving authorities if needed. The interesting point is that it consistently emphasizes consulting professionals, promoting responsible next steps.
- I think the advice is generalized and doesn't account for jurisdiction-specifications, like varying habitability standards or timelines for repairs. Personally, I think the lack of specificity on health risks from mold could delay urgent medical or housing interventions since not everyone have this knowledge.

Question 5: What are the key freedoms protected by the First Amendment, and how do they apply to government actions?

- It breaks down the core freedoms with explanations and examples of government restrictions, making complex constitutional concepts understandable for us. I like the inclusion of exceptions like unprotected speech adds warning issues.
- People might not consider case law or state-specific applications, result in unintended legal violations like social media speech. Relying solely on this might prevent people seeking expert advice for personal situations involving free speech disputes which is not suggested.

Step J - Explore the RAG model (with retrieval)

Now we will ask the **RAG model** the **same questions** as in Step H. Remember that the RAG model is allowed to use the short documents you loaded from the `docs/` folder.

Run the cell below.

Run the RAG model on all questions

```
# @title Run the RAG model on all questions

rag_answers = []
rag_retrieved = []

for i, q in enumerate(questions, start=1):
    print(f"==== RAG Model Answer for Question {i} ====")
    print("Question:", q)
    answer, retrieved_docs = ask_rag_model(q, k=2)
    rag_answers.append(answer)
    rag_retrieved.append(retrieved_docs)

    print("\nAnswer:\n")
    print(textwrap.fill(answer, width=100))
    print("\nRetrieved documents used:")
    for doc in retrieved_docs:
        print(" –", doc["id"], "(", doc["text"][:doc["text"].find("\n")], ")")
    print("\n" + "="*100 + "\n")
```

```
====================================================================================================

==== RAG Model Answer for Question 4 ====
Question: Can I use legal information I find on social media as my main source when making decisions about my case?

Answer:

It's not advisable to use legal information from social media as your main source for making
decisions about your case. While social media can help you learn about legal developments, the
information shared there may oversimplify complex laws or omit critical details relevant to your
situation and jurisdiction.  As the Nolo Legal Encyclopedia suggests, it's important to verify
information with reliable sources, such as official government websites or licensed attorneys. Laws
vary by state, meaning what might apply in one area may not in another. Relying on inaccurate or
generalized information from social media could lead to serious mistakes in legal matters.  For
critical legal issues, consult qualified professionals to get tailored advice based on your specific
circumstances.

Retrieved documents used:
 – law_4 ( Source: Nolo Legal Encyclopedia — Avoiding Misinformation on Social Media )
 – law_2 ( Source: ACLU — Know Your Rights: Students' Rights in School )

====================================================================================================

==== RAG Model Answer for Question 5 ====
Question: What are the key freedoms protected by the First Amendment, and how do they apply to government actions?

Answer:

The First Amendment of the United States Constitution protects several key freedoms from government
interference. These freedoms include:  1. **Freedom of Religion**: It comprises the Establishment
Clause, which prevents the government from establishing a national religion or favoring one religion
over another, and the Free Exercise Clause, which protects individuals' rights to practice their
religion without undue interference from the government.  2. **Freedom of Speech**: This right
allows individuals to express themselves through words or symbolic actions. The Supreme Court
requires the government to provide substantial justification for any regulation of speech content.
There are specific categories of speech, such as obscenity or incitement to violence, that may
receive less protection.  3. **Freedom of the Press**: Similar to freedom of speech, this right
allows for the publication and dissemination of information without government interference. Members
of the media do not have special rights beyond those of the general public concerning freedom of
expression.  4. **Right to Assemble**: This guarantees the freedom to gather peacefully for lawful
purposes. It includes the freedom of association, and the government cannot typically require groups
to disclose their membership, except in certain circumstances.  5. **Right to Petition**: This right
allows individuals to ask the government for remedies regarding wrongs they believe they have
suffered, working in conjunction with the right to assemble.  In terms of how these freedoms apply
to government actions: the Supreme Court has interpreted the First Amendment as applying not only to
Congress but to the federal government as a whole, and through the Fourteenth Amendment, to state
governments as well. The government must generally refrain from actions that interfere with these
rights, and if it seeks to regulate them, it must provide strong justification for such
interference. For example, a small group protesting may not need a permit if they are not blocking
traffic or access to buildings.  If you have specific concerns or situations related to these
freedoms, I encourage you to consult a qualified legal professional for tailored advice.

Retrieved documents used:
 – law_5 ( Source: Cornell Law School Legal Information Institute — First Amendment(https://www.law.cornell.edu/wex/first_amendment) )
 – law_3 ( Source: ACLU — Know Your Rights: Protesters' Rights )

====================================================================================================
```

✏️ Reflection

For **two** of the queries you ran above:

For each, compare the **base** and **RAG** answers:

- How is the RAG answer different from the base answer?
- Does the RAG answer feel **more accurate or grounded**? Why or why not?
- Is there **anything still missing or potentially risky** in the RAG answer?
- Based on this, what do you think the **limits** of RAG are in your chosen domain?

[YOUR NOTES HERE]

Question 1: My landlord hasn't fixed a serious mold problem in my apartment for three months. What legal options do I have?**

- The RAG answer is focused on concepts like "habitable premises" and reporting to local housing authorities, directly retrieved from doc law_1 (e.g., landlords' duty to provide habitable premises). It also integrates a caution from law_4 about misinformation, making the response more tied to the docs. The base answer offers a more comprehensive, step-by-step list drawn from general knowledge.
- Yes, it feels more reliable because it draws explicitly from reliable sources (Cornell Law and Nolo). Unlike the base model's risk of outdated or jurisdiction-agnostic advice. It makes less hallucinations.
- lacks specifics on health risks from mold or urgent steps like temporary relocation, which could be risky if the user delays action thinking the advice is exhaustive.

Question 5: What are the key freedoms protected by the First Amendment, and how do they apply to government actions?**

- The RAG answer is more detailed and structured, breaking down freedoms like the Establishment and Free Exercise Clauses with examples from law_5 (e.g., Supreme Court interpretations, unprotected speech categories), and ties in protest-related assembly

from law_3. The base answer is shorter and more high-level without such specifics or references to clauses and applications via the Fourteenth Amendment.

- RAG feels more accurate because providing explanations (e.g., government justification for interference) that align with authoritative text, making it less generic than the base model's summary.
- I feel RAG misses recent case law developments post-2022 (e.g., any 2023-2025 Supreme Court rulings on social media or assembly during events like protests). Riskily, it could give a false sense of encouraging without evolving interpretations, especially in politically charged areas.
- In the law domain, RAG can't handle highly personalized or jurisdiction-specific queries without broader retrieval. This indicates the need for hybrid approaches with real-time web integration for AI legal tools.

Step K - Conceptual reflection

Answer the following questions in your own words. Be sure to answer all questions.

✏️ 1. Why does RAG matter more in law / healthcare than in some other domains?

In 3–5 sentences, explain why RAG is especially important in your chosen domain (law or healthcare). Refer to at least **one specific example** from your experiments.

[YOUR NOTES HERE]

RAG is especially crucial in the law domain because legal information must be precise in reliable sources to avoid severe consequences like misguided actions leading to lawsuits or rights violations. By retrieving specific documents, RAG helps mitigate hallucinations in base models, ensuring responses align with authoritative texts. For instance, in my experiments with the query on First Amendment freedoms, the base model offered a broad overview, but the RAG version incorporated detailed explanations from the Cornell Law document (law_5). This is important in law, where users might act on advice affecting their freedoms or disputes for speech.

✏️ 2. Imagining Fine-Tuning (No Coding)

Watch this YouTube Short about fine-tuning:

- https://www.youtube.com/shorts/Oo1qKoPRrRs?feature=share

After watching this video:

- Explain in a few sentences what fine-tuning is. How does it differ from RAG?

Even though you did not fine-tune a model in this assignment:

- How might fine-tuning improve the model compared to just using RAG or a system prompt?
- If you could fine-tune this model for your chosen domain (law or healthcare), what specific behaviors would you want it to learn? (For example, being cautious, asking clarifying questions, or flagging when it's uncertain.)
- What potential risks might come with fine-tuning in sensitive areas like law or healthcare?

[YOUR NOTES HERE]

- Fine-tuning is the process of taking a pre-trained large language model and further training it on a smaller, domain-specific dataset to adapt its behavior, such as improving accuracy on niche tasks or aligning outputs with particular styles, tones, or knowledge. This involves updating the model's weights through supervised learning on examples like input-output pairs, allowing it to "learn" patterns inherently rather than just recalling from training. Unlike RAG, which dynamically retrieves external documents at inference time to augment prompts without changing the model itself.
- Fine-tuning could enhance the model by internalizing domain-specific nuances, like legal reasoning patterns in advice-giving, leading to more consistent, efficient responses without retrieving and processing external docs each time. Compared to a system prompt, which merely guides behavior temporarily, fine-tuning make these instructions, improving reliability in complex queries.
- In the law domain, I'd fine-tune the model to learn behaviors like consistently flagging jurisdictional variations (e.g., "This applies to U.S. federal law, state rules may differ"), automatically asking clarifying questions for ambiguous queries (e.g., "Could you specify the state or context of the incident?"), and directing to experts (e.g., "consult one for binding advice"). Also, detect high-risk topics like criminal defense strategies, instead prompting for professional consultation to ensure ethical use in AI legal tools.
- Fine-tuning risks like amplifying biases from the training dataset, such as over-representing certain jurisdictions or demographics, leading to skewed advice that could harm underrepresented users. The model also might generating plausible-sounding but incorrect outputs if the fine-tuning data includes outdated or wrong information. I think once fine-tuned, updating the model for new laws guidelines requires retraining, and it could enable misuse, like generating manipulative legal arguments.

✏️ 3. Should organizations deploy systems like this?

Reflect on the following:

- Should an organization deploy a model like this for public use or learning?
  - What potential benefits do you see (e.g., accessibility, faster guidance, reduced workload for staff)?
  - What potential risks concern you (e.g., misinformation, overreliance, loss of human judgment)?
- What technological or policy improvements would need to happen before deployment becomes safe enough?

[YOUR NOTES HERE]

- I believe organizations should deploy RAG-based systems like this for public use or educational purposes, but only as supplementary resources rather than standalone advisors, especially in high-stakes domains like law. Ensuring they could serve in controlled environments, such as non-profits providing initial guidance or educational platforms teaching legal basics. Full deployment without iterations risks amplifying errors.
  - Key benefits include enhanced accessibility, allowing underserved communities (e.g., low-income individuals facing landlord disputes) to quickly access grounded information without barriers like cost or wait times, as seen in our experiments where RAG pulled relevant docs on tenant rights. It also reduces workload for staff in legal aid organizations by handling routine questions, freeing experts for complex cases.
  - Major risks include misinformation from incomplete or outdated docs, for example, if our RAG database missed cases, users might act on outdated advice, leading to legal flaws. Overreliance is a concern, where users treat AI outputs as definitive (as in the mold query, general steps might delay urgent actions), and then skipping lawyers which is not suggested.
- Technologically, integrate real-time updates via hybrid RAG with web search or APIs for current statutes, improve retrieval precision (e.g., using advanced embeddings or multi-query expansion) to avoid irrelevant docs. For policy, we should mandate transparent disclaimers on limitations, enforce data privacy compliance and establish accountability frameworks like human oversight..

Submission

Make sure you've run all code cells, and make sure you've completed all writing requirements. Once you've done that, proceed with submission by obtaining the notebook PDF (File → Print → Save as PDF) and the file you added to the RAG database. Submit both on Canvas.